2024 International Conference on Emerging Innovations and Advanced Computing (INNOCOMP) | 979-8-3503-7647-0/24/$31.00 ©2024 IEEE | DOI: 10.1109/INNOCOMP63224.2024.00108

Peer To Peer Communication Using Web Sockets And Web RTC

[1]**Vineet Sharma**
Department of Computer Science and Engineering, KIET Group of Institutions, Delhi-NCR, Ghaziabad, India
vineet.sharma@kiet.edu

[2]**Mridul Kumar Pandey**
Department of Computer Science and Engineering, KIET Group of Institutions, Delhi-NCR, Ghaziabad, India
mridul.2024cse1150@kiet.edu

[3]**Jayant Gupta**
Department of Computer Science and Engineering, KIET Group of Institutions, Delhi-NCR, Ghaziabad, India
jayant.2024cse1207@kiet.edu

[4]**Hem Kishan**
Department of Computer Science and Engineering, KIET Group of Institutions, Delhi-NCR, Ghaziabad, India
hem.2024cse1108@kiet.edu

Abstract— In the rapidly expanding information era, the Internet has become an indispensable part of peoples' lives, and in this rapidly globalizing world, communication amongst people is paramount. Users can now access websites and other online resources because of the HTTP (Hypertext Transfer Protocol) request-response protocol, which is used to transfer data over the internet. However, one of its disadvantages is that clients cannot access servers' real-time data. Therefore, real-time communication is not possible with technologies like HTTP streaming, long polling, and polling. As a result, WebRTC, or Web Real-Time Communication, and the Web Socket Protocol were used. The latter enables browser-to-browser sharing of media and data in real time. With the help of this peer-to-peer technology, web browsers can now communicate in real time. Almost all contemporary browsers have this technology, which allows them to talk to each other rather than just web servers. Thus, this paper covers the importance of communicating in real-time, the limitations of traditional web real-time communication, an overview of web sockets, and the differences between HTTP and Web Socket Protocol. The paper then explains the significance of audio and video conferencing in the modern world and provides an overview of Web Real-Time Communication (WebRTC), including its architecture and the use of a web-RTC-based system.

Keywords- Web Sockets , WebRTC (Web Real-Time Communication), Peer-to-peer (P2P) networking, Real-time web communication, Data transfer efficiency, Low-latency communication, Secure data transmission, Video conferencing, Audio conferencing

I. Introduction

Web Sockets- Web Socket enables web pages to communicate back and forth with another server via the Web Socket protocol. The document introduces the Web Socket protocol and creates a two-way channel for interaction that runs over an individual Internet socket. WebSocket's primary goals include minimal overhead, and simple two-way communication implementation for applications running on the Internet. When compared to the widely used Comet and Ajax polling solutions—which simulate two way communication by keeping dual Hyper Text Transfer Protocol links open—it offers a notable reduction in response time and network bandwidth. Considering its features, it becomes the optimal solution for constructing an expandable and live online communication infrastructure.

To initiate a connection between two web clients, a new Web Socket instance must be created and launched with the correct Uniform Resource Locator (URL) that corresponds to the external destination that has to be linked [12]. The ws:// scheme for regular Web Socket connections and the wss:// scheme for secure Web Socket connections are provided . However, web browsers hardly never display addresses that contain wss:// or ws://; instead, they often use http:// and https:// [3]. This variant results from the fact that web sockets share the same ports as HTTP, namely port 80 (HTTP) and port 443 (HTTPS), serving as a proxy over HTTP. Despite this resemblance, Web Socket connections are known to be compatible with common HTTP ports (80 and 443) and are recognised as a protocol that can work with firewalls and proxy servers alike. This acknowledgement highlights their capacity to function with ease over common HTTP ports. In order to examine data transfer across Web Sockets while maintaining their proxy properties, a web browser's network analyser is helpful.

Web RTC-WebRTC-based multimedia apps have garnered increasing interest because of their increased development efficiency and user-friendly nature. It eradicates the necessity for external plug-ins or exclusive software by converting live communication into a standardized capability applicable to any online application through an uncomplicated JavaScript API. When developing interactive multimedia systems with low latency throughout the internet environment, it significantly lowers hardware and technological costs. As of right now, WebRTC support is built into the most recent versions of Firefox, Chrome for Android, and Chrome for PC. The official WebRTC standard, which includes the data transfer protocol, browser API, and other components, is presently being developed.

WebSocket allows binary and text data to be transmitted back and forth between a client and a server in a

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on March 28,2026 at 09:19:03 UTC from IEEE Xplore. Restrictions apply.

message-driven manner. This work investigates the fundamental architecture and associated technologies of WebRTC and proposes a WebSocket-based signaling exchange protocol for WebRTC [7]. This technique makes it easier to negotiate multimedia sessions at first between different browsers and encourages the creation of interactive relationships.
The seamless integration of WebSocket and WebRTC in this scenario greatly facilitates signaling information exchange. The study opens the door to smooth negotiation procedures and interactive connections between various browser environments by integrating WebSocket as the signaling carrier. This innovative method demonstrates how WebSocket and WebRTC can be combined to create versatile and effective multimedia communication systems. Finally, by utilizing this protocol for communication and the power of the WebRTC API, a real-time interactive multimedia system that functions in a mobile internet setting was designed and built. Mobile devices or desktop computers can be the peers in this system.

II. Exploring Challenges In Traditional Real-Time Communication Methods

Traditionally, techniques like polling, extended polling, and HTTP streaming have been used to mimic a continuous connection between the client and the server in order to enable real-time communication over the web. The shortcomings of each of these approaches are addressed by Web Sockets.

Polling:
By using the polling technique, the client periodically sends HTTP requests to the server to inquire about updates. Polling can result in a large number of requests, the majority of which return no new data, increasing server load and needless network traffic [1]. Updates are only obtained at the polling frequency interval, which results in a delay in the acquisition of new data—not ideal for real-time applications. Resource Consumption: A complete HTTP request and response cycle is involved in each poll, which may require a lot of resources.

Long Polling:
Compared to traditional polling, which involves the client submitting a request to the server, long polling allows the server to hold onto the request until fresh data becomes available. The cycle continues when the client sends a new request as soon as it receives the updated data. Because of the requirement to manage timeouts and multiple simultaneous connections, long polling can be difficult to manage. Server Load: Keeping connections open for several clients may result in a higher demand on the server's resources. User experiences may be inconsistent due to response times that fluctuate based on network conditions and server processing times.

HTTP streaming:
Through a single HTTP connection that stays open, the server can send data to the client continuously using HTTP streaming. After receiving data, the client maintains the connection, enabling a unidirectional stream from server to client. Although data is sent from the server to the client continuously through HTTP streaming, data sent from the client to the server requires a separate HTTP request [1]. Compatibility problems may arise because not all web browsers or network intermediaries support HTTP streaming. Streaming still has the overhead of HTTP headers with every data transmission, just like other HTTP-based methods.

Web Sockets create a full-duplex communication channel over a single, persistent connection, thereby overcoming the drawbacks of the conventional techniques. Without having to create new connections or include extra HTTP headers with every message, data can be sent directly between the client and server once a WebSocket connection has been established [1]. Near real-time communication is made possible by Web Sockets, and this is essential for interactive applications like chat services and online gaming. The data frames that are transferred between the client and server after the initial connection handshake are brief and light, which uses less bandwidth and overhead. Because Web Sockets allow two-way communication, servers can instantly send updates to clients without having to wait for a request from a client. There is less need for repeated handshakes and connection setups thanks to the WebSocket protocol, which maintains a persistent connection.

III. Comparison of HTTP and WEB SOCKETS

HTTP is a stateless protocol, every request made by a client to a server is handled as a separate transaction that is unconnected to any requests made earlier. Since certain applications need a continuous state of interaction, this could result in inefficiencies. The foundation of HTTP is the request/response paradigm, in which a request is sent by the client and answered by the server [4]. Although this model is not optimized for real-time communication, it is perfect for document retrieval. Traditionally, HTTP is a unidirectional protocol, meaning that the client initiates communication. Without first receiving a request, the server is unable to send messages to the client.
Headers and other protocol-specific data are included in every HTTP request/response pair. These can add a lot of overhead, particularly when small amounts of data are being transferred frequently. HTTP responses contain headers indicating the server software, content type, length, and set cookies for storing client-side data. In HTTP requests, headers convey information such as the user agent's browser details, preferred language, accepted encodings, and session-related data like cookies. Using polling can result in significant network traffic spikes, bandwidth usage and it also caused delays due to time consumed in inspecting the headers. This displayed growing awareness of the need to investigate alternative real-time communication systems in the future due to the shortcomings of polling [2].

A persistent connection between the client and the server is made possible by Web Sockets. This implies that a WebSocket connection is open for two-way communication

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on March 28,2026 at 09:19:03 UTC from IEEE Xplore. Restrictions apply.

once it is formed and stays open until it is closed by the client or the server. Full-duplex communication channels, which enable simultaneous data transmission in both directions, are supported by Web Sockets. This is especially helpful for real-time applications that need instantaneous and continuous data transfer, like chat, gaming, and live streaming. The purpose of Web Sockets is to minimize overhead. Compared to HTTP, there is a significant reduction in the additional data overhead for messaging after the initial handshake to establish the WebSocket connection, which makes it more efficient for short and frequent messages[9].

Web Socket connections have fewer control information in their headers. However, using Cookie and Session in the header is prohibited by the specifications of the Web Socket protocol. User data only has two UTF-8 encoded control bits attached to it: "\x00" at the start and "\xFF" at the finish. Web Sockets reduce network traffic and latency by drastically reducing bandwidth and processing time for handling headers.
Bidirectional communication is made possible by Web Sockets, which eliminates the need for the client to make requests in order for the server to communicate with the client at any time. Applications that require real-time client updates, like stock tickers or notifications, must have this feature.

When communication is primarily one-time or sporadic, like when retrieving web pages, images, and data, HTTP is a good choice for transactional requests. Conversely, Web Sockets are intended for situations requiring continuous, interactive communication. Web Sockets are more effective for applications that need real-time updates and low latency because of their lower overhead and persistent connection. Because a new connection must be made for every request, HTTP may be less effective in certain use cases. Because Web Sockets require stateful interactions and the maintenance of a persistent connection, their implementation can be more complicated than that of HTTP. However, the advantages of real-time communication frequently outweigh this complexity.

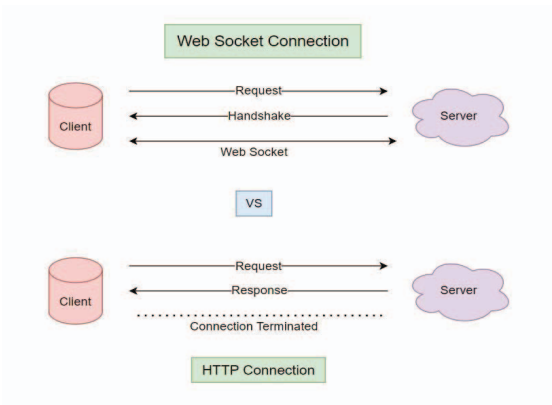

Figure 1- Comparison of Web Socket and HTTP

IV. Web Rtc Architecture

Instantaneous communication features inside web browsers are made possible by the core architecture of WebRTC, which does not require additional plugins or software installations. The getUserMedia API for accessing user audio and video devices, the RTCPeerConnection API for establishing direct peer-to-peer communication, and the RTCDataChannel API for facilitating data transmission between browsers are the three main components that make up WebRTC. Together, these APIs make it easier to create web apps that allow data, audio, and video communication in real-time, which makes WebRTC an essential technology for a variety of online platforms that promote collaboration and communication. WebRTC architecture has a strong focus on decentralized, peer-to-peer communication, which is an important feature. WebRTC allows direct contact between browsers, enabling decreased latency and improved privacy, in contrast to traditional communication systems that rely on centralized servers to conduct communication. This decentralized structure is especially important in situations like online gaming, video conferencing, and collaborative document editing where real-time communication is essential. Peer-to-peer communication optimizations, scalability issues, and the effect of changing network circumstances on the overall performance of real-time web applications are all common topics of research in WebRTC architecture.

One of the main priorities in WebRTC architectural research is security. Since WebRTC allows browsers to communicate directly with one another, it is crucial to protect the integrity and confidentiality of data that is transferred. WebRTC security protocols are essential and include encryption methods such as Datagram Transport Layer Security (DTLS) to guarantee media transmission security and Secure Real-Time Transport Protocol (SRTP) to protect audio and video content [2]. Ongoing efforts to provide reliable and safe WebRTC implementations include looking into possible vulnerabilities, improving encryption techniques, and researching defenses against attacks. A study in this field would explore these security features and highlight developments that reinforce WebRTC's standing as a reliable and safe real-time communication tool.

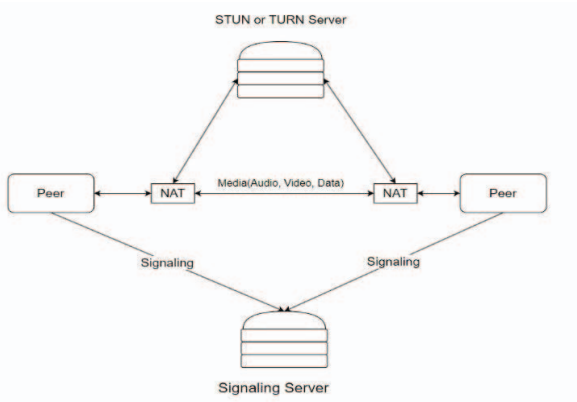

Figure 2 - WebRTC Architecture

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on March 28,2026 at 09:19:03 UTC from IEEE Xplore. Restrictions apply.

V. Web Rtc Api

A robust collection of interfaces and protocols known as Web Real-Time Communication (WebRTC) API makes it possible to conduct real-time communication right within web browsers. There are three primary parts that make up the WebRTC API. Web applications can access a user's audio and video devices through the getUserMedia API, providing services like live streaming and video conferencing. By facilitating the creation of peer-to-peer communication channels between browsers, the RTCPeerConnection API makes it possible to communicate audio and video data without the use of middle-man servers. Finally, the RTCDataChannel API makes WebRTC more versatile for a wider range of interactive applications by enabling the transfer of arbitrary data across browsers.

Getting multimedia content from a user's device and incorporating real-time audio and video streams into apps is made possible by the getUserMedia API. Online gaming, interactive multimedia apps, and video conferencing platforms have all found great use for this API. In the meantime, the RTCPeerConnection API manages the negotiation and setup of peer connections while optimizing for parameters like bandwidth and latency, guaranteeing that direct communication between browsers is effective and secure. By enabling browsers to transmit non-media data, the RTCDataChannel API enhances these functionalities and promotes the development of more dynamic and interactive online applications.

One notable benefit is the WebRTC API's intrinsic ability to function independently without the need for plugins or other external third-party software [10]. Web browsers have integrated functionality that improves accessibility and user experience and makes it easier for developers to incorporate real-time communication features.WebRTC is a typical option for developing real-time communication apps on the web due to its widespread adoption and support by major browsers. Its open nature further contributes to this. Research in this field frequently looks at security issues, optimization strategies, and the continuous development of the WebRTC API to satisfy the many demands of consumers and developers in the dynamic web technology environment.

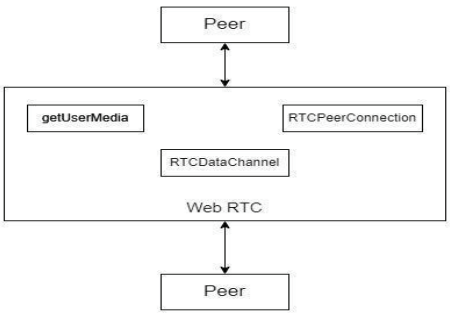

Figure 3- WebRTC API

VI. Ensuring Secure Communication in Web Sockets and WebRTC

Security is a top priority in the world of web technologies, especially for protocols like Web Sockets and WebRTC, which allow for real-time communications. Web Sockets, while allowing for a persistent connection between the client and server, introduce potential security risks such as Cross-Site WebSocket Hijacking (CSWSH), in which attackers can send requests through a user's browser without their knowledge. Furthermore, the risk of data interception exists if the data transfer is not encrypted, and denial-of-service (DoS) attacks pose a significant threat to service availability.

To address these issues, using WebSocket Secure (WSS) is critical because it encrypts data transmission, protecting against eavesdropping and tampering. Furthermore, integrating token-based authentication ensures that only authorized users can initiate WebSocket connections, and rate limiting, combined with careful connection management, can reduce the risk of DoS attacks.

WebRTC, which was designed for direct peer-to-peer communication, also has security flaws. Man-in-the-middle (MitM) attacks can occur if communication is not properly encrypted, allowing for potential eavesdropping. Identity hijacking during the signaling process is another issue that can lead to unauthorized access. Furthermore, WebRTC has the potential to reveal a user's true IP address, harming privacy even when using virtual private networks (VPNs).

To improve WebRTC security, secure signaling mechanisms are required. Unauthorized access can be prevented by using secure channels for signaling, such as HTTPS, and authenticating all communication parties. Implementing Datagram Transport Layer Security (DTLS) is mandatory because it ensures that the media stream is encrypted. Configuring STUN and TURN servers properly can help mask users' IP addresses, thereby protecting their privacy.

VII. Proposed System

The proposed system streamlines the user journey, starting with the invitation and registration procedures. By utilizing WebRTC and Web Sockets, users may easily move from starting video calls and messaging to ending sessions. The user-friendly interface of the flow chart guarantees effective communication by enabling smooth beginning, stopping, and ending of contacts.

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on March 28,2026 at 09:19:03 UTC from IEEE Xplore. Restrictions apply.

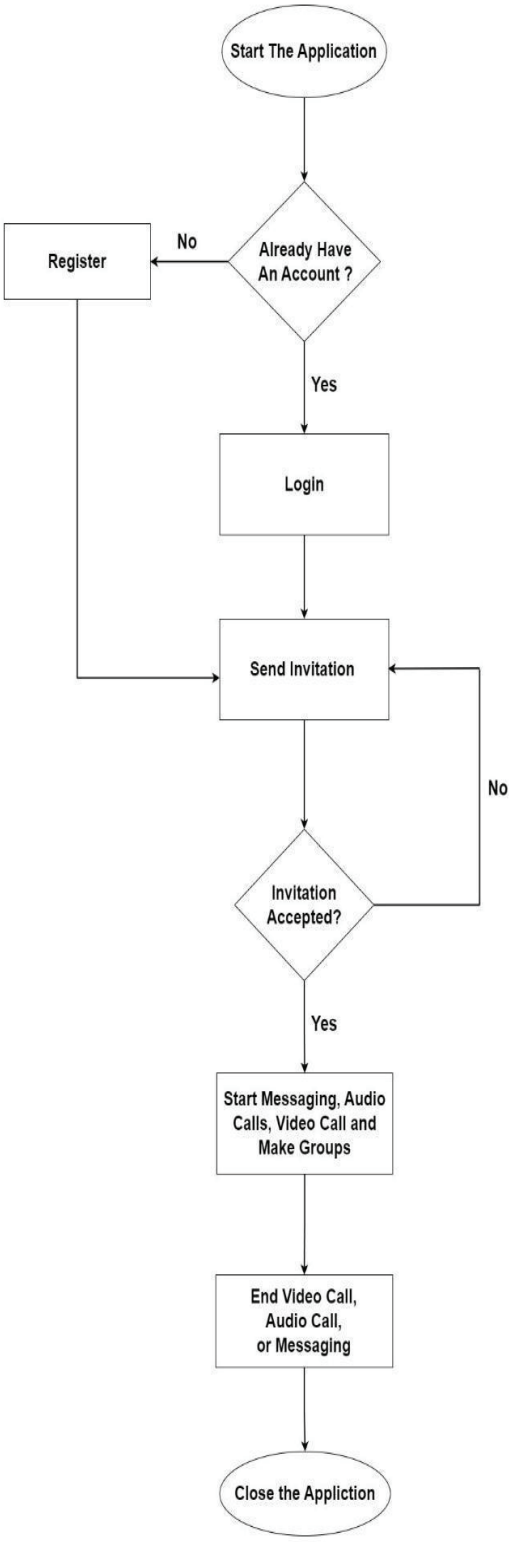

Figure 4 – FlowChart

VIII. IMPLEMENTATION

Register Page

The initial step for users to join the platform is to visit the registration page. Users provide their email address, a strong password, and a distinctive username. To submit the information, click the "Register" button. For security purposes, it is imperative to assist users in creating strong passwords. Correct submissions are ensured by validation checks, and terms of service agreement checkboxes may be optionally inserted. The application's group chat, voice call, and video call functions are accessed through this page.

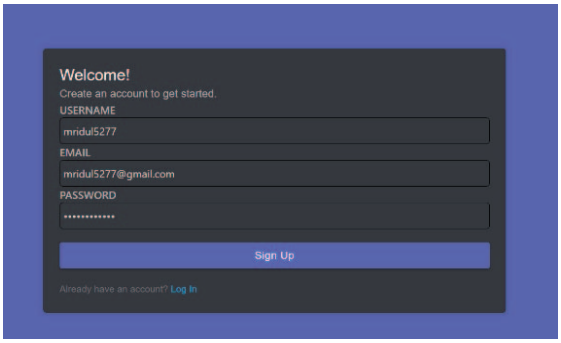

Figure 5 - Register Page

Home Page

The main page serves as a focal point and offers users all the tools they need to communicate and engage with others. Within the site, friends can be managed and viewed by users, encouraging social ties. Furthermore, group creation facilitates cooperative conversations and activities. Users are kept aware about incoming social interactions by the existence of new friend requests and message notifications, which adds to a dynamic and interesting user experience. The platform's general usability and appeal are improved by the design, which places a strong emphasis on real-time updates and user-friendly navigation.

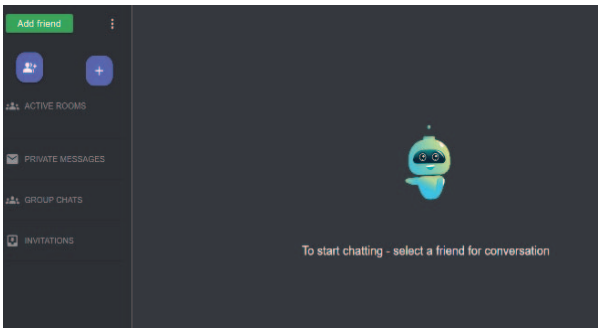

Figure 6 - Home Page

Invitation Page

An essential element that makes it easy for people to connect and start conversations is the Invitation Page. Usually enabled by a button or form, this method entails a

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on March 28,2026 at 09:19:03 UTC from IEEE Xplore. Restrictions apply.

user-friendly interface where the sender enters the friend's username or chooses from already-existing connections. The user is immediately informed on the Invitation Page upon accepting an invitation, as seen in the example where "anunjay" accepts an invitation from "mridul5277". The effective establishment of a link between the two users is indicated by this approval. Notably, WebSockets integration makes this connection even more real-time by enabling rapid notifications and updates without requiring frequent page refreshes. All things considered, the platform's Invitation Page is essential for building user relationships and starting deep dialogues.

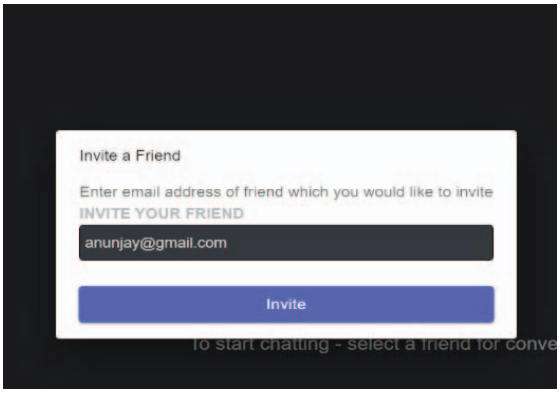

Figure 7- Invitation Send

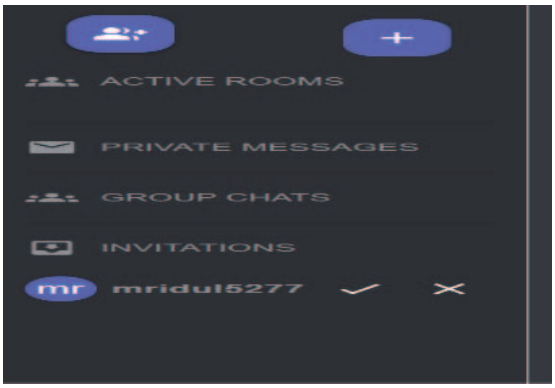

Figure 8- Invitation Received

Video Call

An example of how the Video Call feature smoothly incorporates people into in-person discussions is the exchange between "mridul5277" and "anunjay." This feature allows for real-time video conversation right within the web browser and is made possible by WebRTC technology. Users can participate in immersive video conversations with one another after receiving an invitation, which promotes a more intimate and dynamic communication experience. A smooth and responsive connection is created between the participants thanks to WebRTC, which guarantees the effective transfer of audio and video streams. Peer-to-peer technology improves video call security and privacy while also facilitating a dependable and easy-to-use experience.

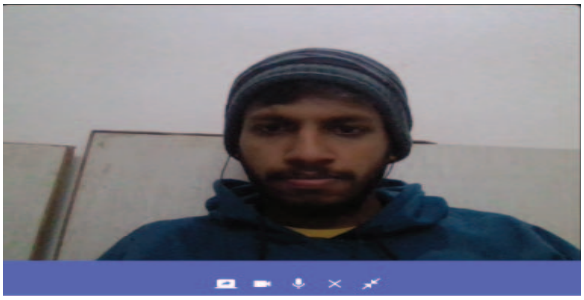

Figure 9- Video Call (User1)

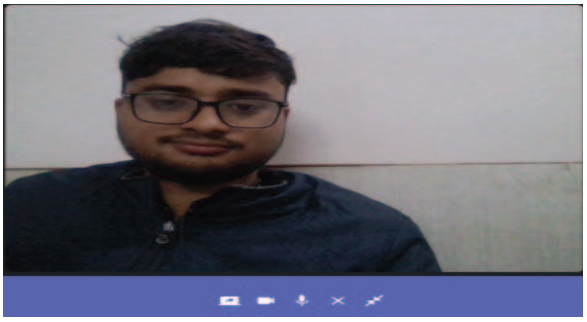

Figure 10- Video Call(User 2)

Group Chat

One example of how the Group Chat function improves teamwork on the platform is when "mridul5277" creates a group called "Buddies" and invites his friend "anunjay." This feature allows group members to communicate textually in real time using WebSockets technology. Users can form groups, give them names, and invite others to join, creating a communal forum for conversation. As exemplified, "mridul5277" and "anunjay" converse in a smooth and accommodating group chat within the "Buddies" group that they have formed. Instant message delivery is ensured by the use of WebSockets, enabling dynamic and engaging group discussion. Within designated groups, the Group Chat function fosters community building, information sharing, and conversation among users, giving the platform a more social aspect.

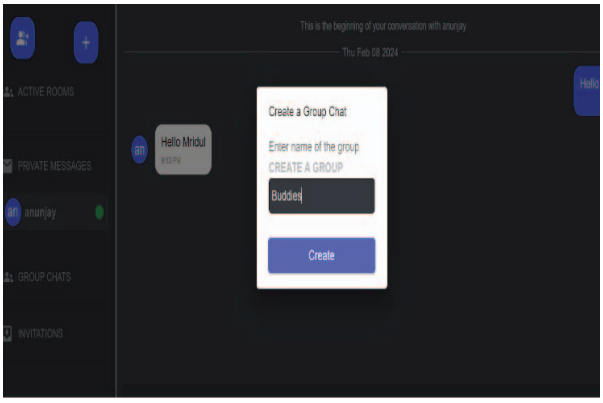

Figure 11- Creating Group

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on March 28,2026 at 09:19:03 UTC from IEEE Xplore. Restrictions apply.

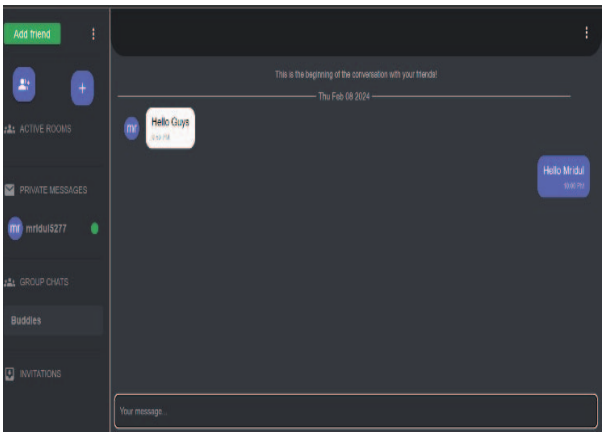

Figure 12- Group Chat

IX. Performance Evaluation of the system: Web Sockets vs. HTTP

In a comparative study on the performance of HTTP and Web Sockets via Socket.io, we used browser-based tests that excluded the initial WebSocket connection setup time. The findings indicated that Web Sockets consistently outperformed HTTP in terms of latency and throughput. On average, an HTTP request took 105ms to complete, whereas a Socket.io request took only 82ms. Notably, Socket.io processed 50 parallel requests in about 170ms, whereas HTTP took approximately 5 seconds due to browser-imposed concurrent connection limits. Furthermore, while HTTP could only handle 10 requests per second, Socket.io handled nearly 4000, demonstrating its superior ability to handle large volumes of traffic.

The efficiency of data transfer also favored Web Sockets, with HTTP consuming 262 bytes per request-response cycle versus Web Sockets' lean 52 bytes. This stark contrast in data usage demonstrates the advantages of Web Sockets, particularly in applications that require frequent data exchanges.

Web Sockets' benefits were especially noticeable in real-time chat applications. Messages sent via Web Sockets arrived with an average latency of less than 100ms, allowing for nearly instant communication. In comparison, HTTP polling had latencies of more than 500ms. Web Sockets' persistent connection enabled a smooth and continuous message flow, improving the user experience while reducing the strain on server resources. The removal of repeated HTTP headers from each message improved performance even further. Our findings clearly show that Web Sockets, particularly Socket.io, offer a responsive and efficient solution for real-time communication that outperforms traditional HTTP in all tested areas.

X. CONCLUSION AND FUTURE WORKS

The purpose of using traditional communication methods for real-time communications was initially discussed in this study. Next, it examined the fundamental ideas of Web RTC and Web Sockets and how they can address those problems. The study examined WebRTC's architecture and implementation in detail. Web Sockets are a cutting-edge technology that are quickly becoming the best option for developing real-time communication systems. WebRTC technology facilitates peer-to-peer communication across the globe. WebRTC's strong data transmission capabilities and secure real-time communication have made it possible to develop webpages and applications that enable user participation in peer-to-peer or group connections. Real-time file sharing, messaging, and audio/video conferencing are made possible by this innovation.

A peer-to-peer learning system that uses WebRTC has been effectively developed and used in the educational space. This structure encourages participation, cooperation, and communication inside and outside the conventional classroom. WebRTC's affordability and lack of plugin requirements have increased learning's accessibility and reach. Peer-to-peer architecture also offers scalability to support numerous users concurrently, potentially saving money on infrastructure and network expenses. It would also be beneficial to investigate the system's possible applications in non-educational fields. Hospitals, businesses, and organizations can all benefit from this approach to increase customer satisfaction. For example, real-time collaboration can be facilitated by a free and user-friendly tool that makes it easy for busy people to share papers and information [8].

In summary, the Web Socket and WebRTC Peer-to-Peer Communication technology not only transforms education but also offers prospects for extensive implementation in various sectors. We can guarantee the ongoing relevance and enhancement of this cutting-edge communication solution by attending to these future directions.

XI. REFERENCES

[1] Qigang Liu, Xiangyang Sun, "Research of Web Real-Time Communication Based on Web Socket," Sydney Institute of Language & Commerce, Shanghai University, Shanghai, China. Email: ryan.liu@shu.edu.cn. Received September 4, 2012; revised October 15, 2012; accepted October 26, 2012.

[2] H. Fateh Ali Khan, A. Akash, R. Avinash, C. Lokesh, "WebRTC Peer to Peer Learning," Department of Information Technology, Valliammai Engineering College, Chennai, India. Published by: International Journal of Engineering Research & Technology (IJERT), Vol. 7 Issue 03, March-2018. http://www.ijert.org ISSN: 2278-0181. IJERTV7IS030135. (This work is licensed under a Creative Commons Attribution 4.0 International License.)

[3] Zinah Tareq Nayyef, Sarah Faris Amer, Zena Hussain, "Peer to Peer Multimedia Real-Time Communication System based on WebRTC Technology," Department of Computer Science, Dijlah University College. Published in International Journal of Engineering & Technology, 7 (2.9) (2018) 125-130.

[4] WenPeng Wang, Lingli Mei, "A design of Multimedia Conferring System Based on WebRTC Technology," Boston University, IEEE, 2017.

[5] Zafran M RM, Gunathunga L G K M, Rangadhari M I T, Gunathne M D D J, Kuragala K R S C B,

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on March 28,2026 at 09:19:03 UTC from IEEE Xplore. Restrictions apply.

MrDhishanDhammearatchi, "Real Time Information and communication Center based on WebRTC," International Journal of Science and Research Publication, volume 6, Issue 4, ISSN 2250- 3153, April 2014.

[6] IgorDujlovic, Zoran Duric, "Cross-Platform Web-based Real-time communication in Web TV and Video on Demand System," 57th international symposium, Zadar Croatia ELMAR-2015, 28-30 September 2015.

[7] HuayingXue, Yuan Zhang, "A WebRTC-Based Video Conferencing System with Screen Sharing," IEEE International Conference on Computer and Communication, 2016.

[8] Shi Yuzhuo, Hao Kun, "Design and Realization of Chatting Tool Based on Web," IEEE, 2013.

[9] WajdiElleuch, "Models for Multimedia conference between Browsers based on WebRTC," the sixth international workshop on selected topics in mobile and wireless computing, IEEE, 2013.

[10] P. Mohamed Shakeel, S. Baskar, V. R. Sarma Dhulipala, S. Mishra, and M. M. Jaber, "Maintaining Security and Privacy in Health Care System Using Learning Based Deep-Q-Networks," J. Med. Syst., 2018.

[11] S. K. Abd, S. A. R. Al-Haddad, F. Hashim, A. B. H. J. Abdullah, and S. Yussof, "Energy-Aware Fault Tolerant Task offloading of Mobile Cloud Computing," in Proceedings - 5th IEEE International Conference on Mobile Cloud Computing, Services, and Engineering, MobileCloud 2017, 2017.

[12] Alok Dubey, "Enhancing Real Time Communication and Efficiency With Websocket," Satish Pradhan Dnyansadhna College, Thane, Maharashtra, India. Published in International Research Journal of Engineering and Technology (IRJET), Volume: 10 Issue: 08 | Aug 2023. www.irjet.net p-ISSN: 2395-0072.

[13] S. Loreto and S.P. Romano, "How far are we from WebRTC-1.0? An update on standards and a look at what's next," IEEE Communications Magazine, vol. 55, no: 7, May 2017, pp. 200-207, DOI: 10.1109/MCOM.2017.1600283.

[14] Felix Weinrank, Martin Becke, et al, "WebRTCData Channels," IEEE Communications Standard Magazine, vol. 1, no. 2, July 2017, pp. 28-35, DOI: 10.1109/MCOMSTD.2017.1700007.

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on March 28,2026 at 09:19:03 UTC from IEEE Xplore. Restrictions apply.